

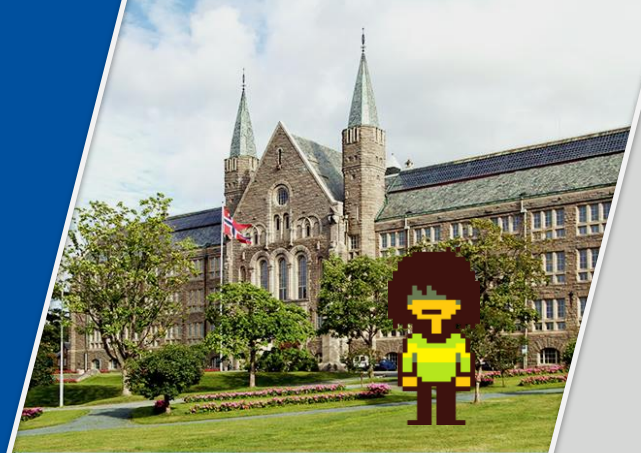


Norwegian University of
Science and Technology

Assignment Lecture 1

Search Algorithms

TDT4137 – Introduction to Artificial Intelligence, 2026
Aleksandra Jekic, Department of Computer Science



Relevant for this assignment

- Lecture 2 (yesterday) and Lecture 3, which covers some mathematical theory on A^* relevant to this assignment (next week)
- Chapter 3 in the Russell and Norvig book (4th edition, Global).

If you have taken TDT4120 Algorithms and data structures, some of this may be familiar to you and you will be used to this way of thinking. However, you can also get tricked. Even small, subtle differences in our versions of the algorithms can be significant.

Note also that older versions of the book have different algorithms.

How important is this?

- If you look at the old exams, there is usually a lot on this topic.
- May be easy to grasp intuitively, but small details can trip you up. Behind the scenes we joke that we are still discovering new subtle details.
- In the exam, you may have to do these algorithms by hand. Most human brains require spaced repetition to do this confidently. This assignment is a good chance to build a solid foundation for your exam practice period.

What version to use?

- Many different algorithms with many different properties have the same names. If you look up random A^* content online, it is probably not our A^* .
- Small details that can make a really big difference (from main lectures):
 - Does the algorithm save where it has been (graph search) or not (tree search)?
 - When does the algorithm check if it found the goal?
 - How does the algorithm generate the path to the goal after finding it?

What version to use?

- We have tried to make this as unambiguous as possible.
- For Breadth-first-search (BFS), the official algorithm is the one found on page 95 in Norvig and Russell.
- Biggest “gotcha” is that it checks for the goal immediately when discovering a new node.

```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure
  node ← NODE(problem.INITIAL)
  if problem.IS-GOAL(node.STATE) then return node
  frontier ← a FIFO queue, with node as an element
  reached ← {problem.INITIAL}
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if problem.IS-GOAL(s) then return child
      if s is not in reached then
        add s to reached
        add child to frontier
  return failure
```

gn g
have J
explored

What version to use?

- We have tried to make this as unambiguous as possible.
- For all the other algorithms, please use Best-First-Search on page 91 with the appropriate function f to order the priority queue.

```

function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node ← NODE(STATE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s] ← child
        add child to frontier
  return failure

```

```

function EXPAND(problem, node) yields nodes
  s ← node.STATE
  for each action in problem.ACTIONS(s) do
    s' ← problem.RESULT(s, action)
    cost ← node.PATH-COST + problem.ACTION-COST(s, action, s')
    yield NODE(STATE=s', PARENT=node, ACTION=action, PATH-COST=cost)

```

Depth-First Search

$$f(n) = -\text{depth}(n)$$

Uniform-Cost Search (Dijkstra)

$$f(n) = g(n)$$

Greedy Best-First Search

$$f(n) = h(n)$$

A* Search

$$f(n) = g(n) + h(n)$$

What version to use?

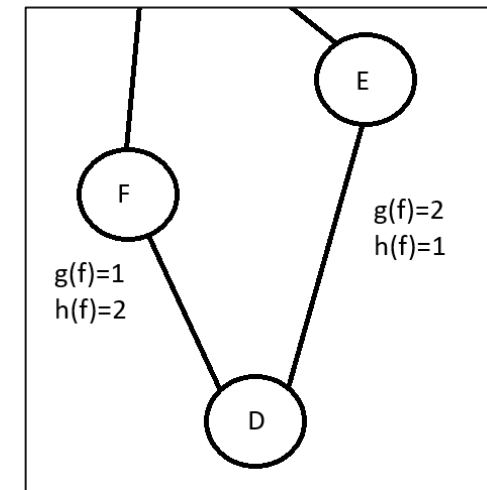
- Process nodes in alphabetical order, when not specified.
- If function f , used to order the priority queue, has a tie, fall back on alphabetical order.

Assumptions

- Use the pseudocode from the book:
 - For 1 (BFS), use `Breadth-First-Search`, from **Figure 3.9 (p. 95)**
 - For 2–5, use `Best-First-Search`, from **Figure 3.7 (p. 91)** with the following heuristic functions:
 - DFS: use $f(n) = -\text{depth}(n)$
 - Uniform-Cost Search: use $f(n) = g(n) = \text{PathCost}(n)$
 - Greedy Best-First Search: use $f(n) = h(n)$
 - A^* : use $f(n) = g(n) + h(n)$
 - Process (or generate) nodes in alphabetical order
 - So, C gets processed before H when `Expand( $B$ )`
 - In case of a tie, use alphabetical order as a tie breaker
 - E.g., G will be at the top of the stack when `Expand( $A$ )` on DFS, although G and I have the same depth

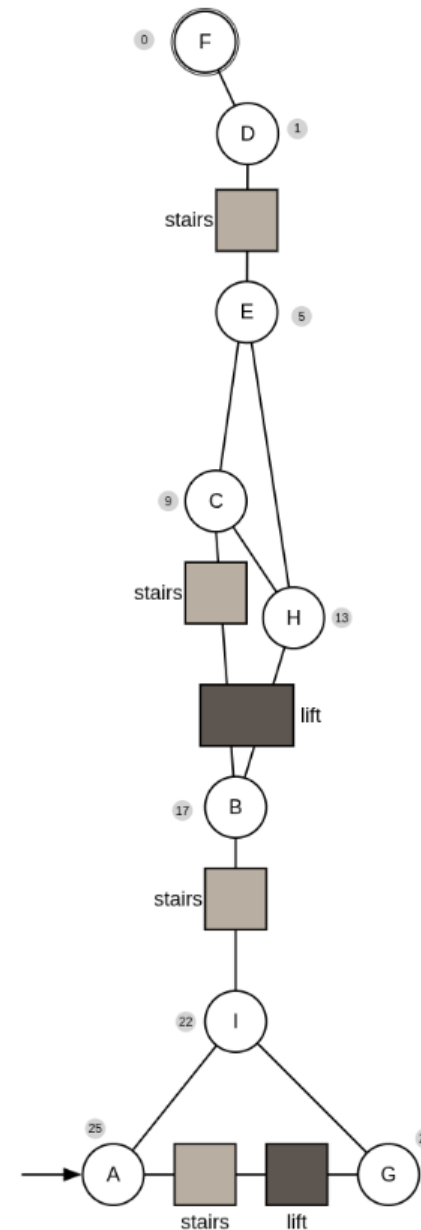
Implications of using the pseudocode

- By using the pseudocode directly, we avoid "graph-search"- or "tree-search"-like terms
- We don't have to specify that "BFS has an early goal check" as it is clearly described in the algorithm
 - `Best-First-Search` has a late goal check, and we don't have to specify it either
- In `Best-First-Search`, if we find cheaper path to a node, it may be added into `Frontier` more than once. However, since there can only be one of each node in `Reached`, the second expansion will make no difference. To keep the solution easy to read, we do not include these superfluous expansions.
- When we `Expand` a node, we check **all descendants** (including nodes "seen" before) as we are traversing a graph
 - In `Breadth-First`, we **only add** them to the data structures `Reached` and `Frontier` if they are not in `Reached`
 - In `Best-First`, we **only add** them to the data structures `Reached` and `Frontier` if:
 - the node is not in `Reached`, or
 - the node has a cheaper `PathCost` from this node
 - We **never** add the parent node (where we came from) since the `Node` data structure contains it as an attribute
 - The **parent** of a node n is the node which was expanded when n was added to `Reached`
 - This also includes 'overwrites', when a node has a cheaper `PathCost`



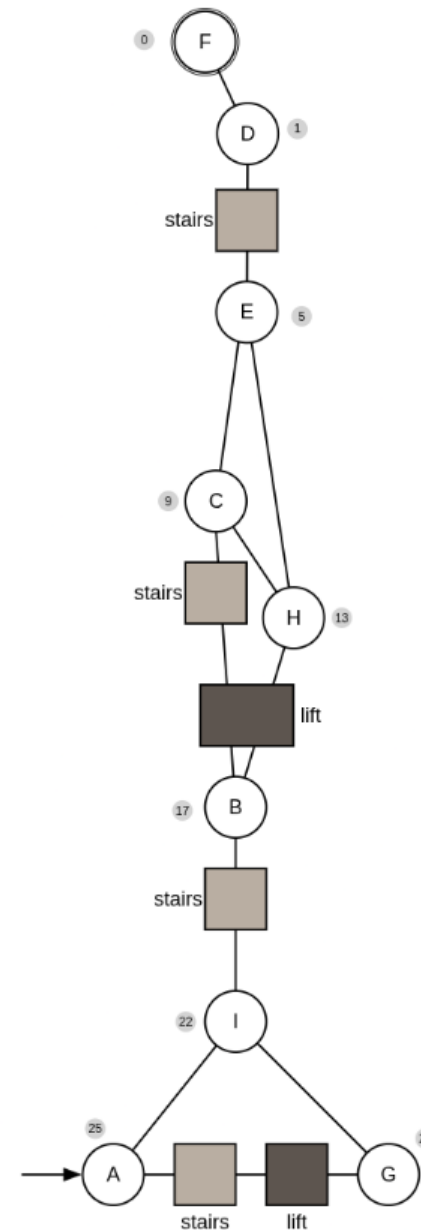
Assignment 1

- Task: We want to impress a boy/girl/something by playing through the Phobos Anomaly level in the video game Doom (1993) as fast as possible.



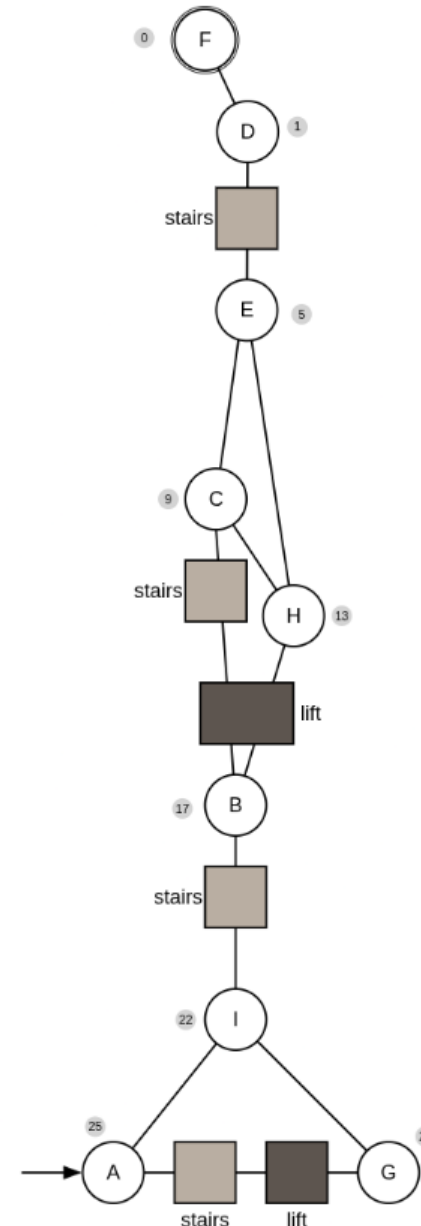
Assignment 1

- Task: We want to impress a boy/girl/something by playing through the Phobos Anomaly level in the video game Doom (1993) as fast as possible.
- A is start. F is the goal. The numbers in the gray circles is the heuristic value.
- Crossing any line costs 1. Stairs cost extra 1. Lift costs extra 2.
- Going from A to G costs $1+1+2=4$.
- First task is just to write the cost of all possible paths so that we have them handy. (A, G) = 4, etc.



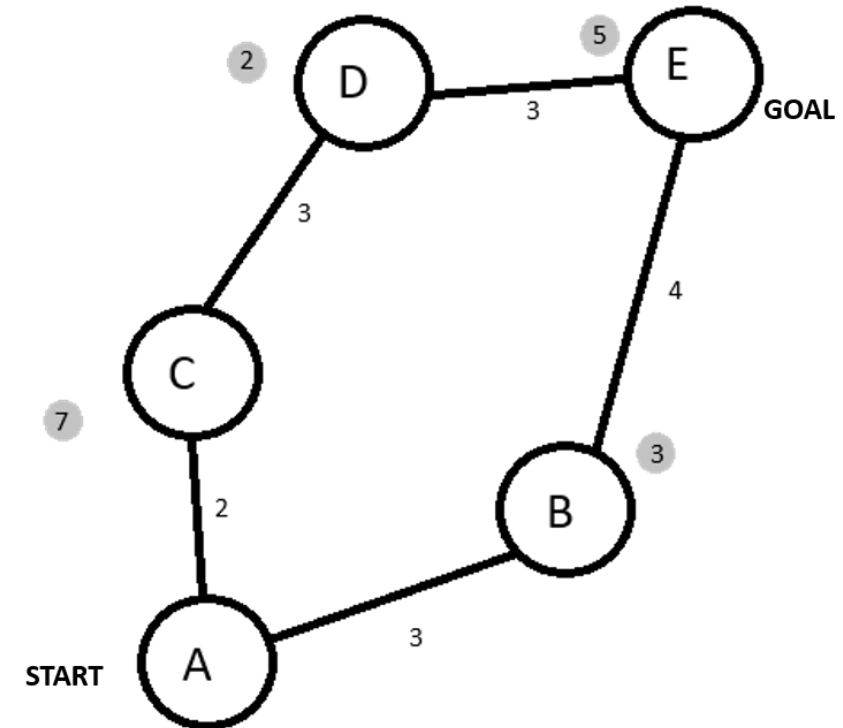
Assignment 1

- The next part is to solve this manually with the following algorithms:
 - Breadth-first search (BFS)
 - Depth-first search (DFS)
 - Uniform-cost search/Dijkstra (UCS)
 - Greedy best-first search (GBFS)
 - A* search
- Example of solution:
 - Order of expansion: S, A, B, (G)
 - Found path: SBG
 - Path cost: 4



Assignment 1

- Not going to spoil the Doom graph for you, but let's see if we agree how this works.
- Breadth-First Search (BFS).
- Solution:
 - Order of expansion: A, B, (E)
 - Found path: A, B, E
 - Path cost: 7



```

function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure
  node ← NODE(problem.INITIAL)
  if problem.IS-GOAL(node.STATE) then return node
  frontier ← a FIFO queue, with node as an element
  reached ← {problem.INITIAL}
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if problem.IS-GOAL(s) then return child
      if s is not in reached then
        add s to reached
        add child to frontier
  return failure
  
```

gn g
have J
explored

- Example of solution:
 - Order of expansion: S, A, B, (G)
 - Found path: SBG
 - Path cost: 4

Assignment 1

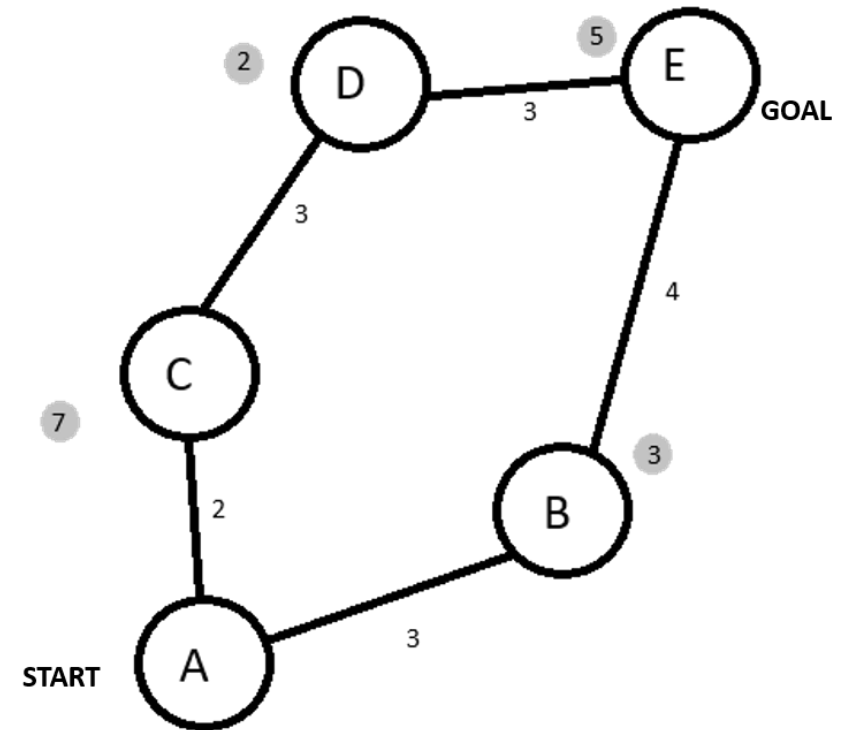
- Not going to spoil the Doom graph for you, but let's see if we agree how this works.
- Depth-first search (DFS).
- Solution:
 - Order of expansion: A, B, (E)
 - Found path: A, B, E
 - Path cost: 7

```

function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node ← NODE(STATE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s] ← child
        add child to frontier
  return failure
  
```

```

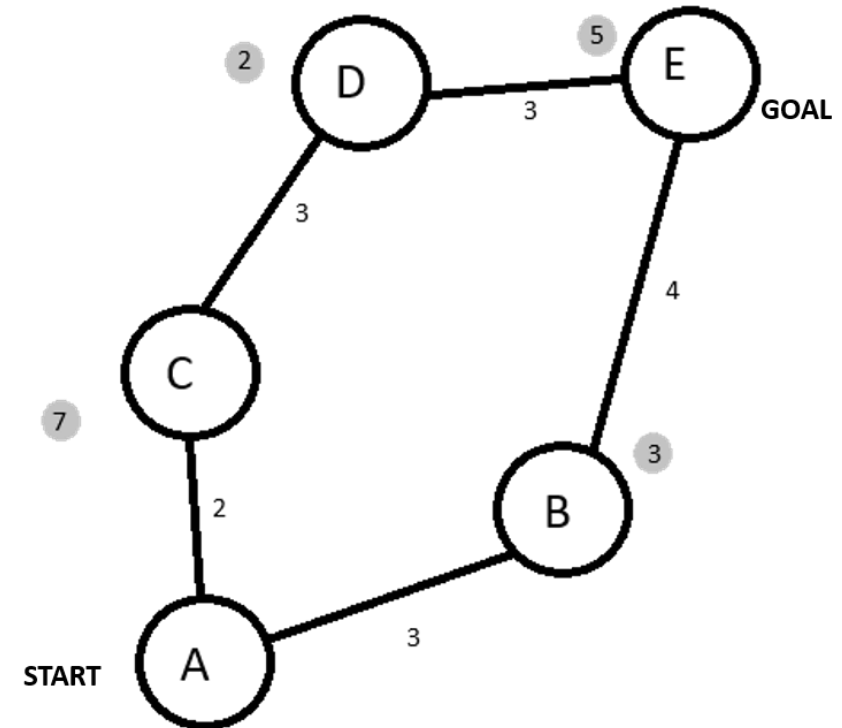
function EXPAND(problem, node) yields nodes
  s ← node.STATE
  for each action in problem.ACTIONS(s) do
    s' ← problem.RESULT(s, action)
    cost ← node.PATH-COST + problem.ACTION-COST(s, action, s')
    yield NODE(STATE=s', PARENT=node, ACTION=action, PATH-COST=cost)
  
```



- Example of solution:
 - Order of expansion: S, A, B, (G)
 - Found path: SBG
 - Path cost: 4

Assignment 1

- Not going to spoil the Doom graph for you, but let's see if we agree how this works.
- Uniform-cost search/Dijkstra (UCS).
- Solution:
 - Order of expansion: A, C, B, D, (E)
 - Found path: A, B, E
 - Path cost: 7



- Example of solution:
 - Order of expansion: S, A, B, (G)
 - Found path: SBG
 - Path cost: 4

```

function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node ← NODE(STATE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s] ← child
        add child to frontier
  return failure
  
```

```

function EXPAND(problem, node) yields nodes
  s ← node.STATE
  for each action in problem.ACTIONS(s) do
    s' ← problem.RESULT(s, action)
    cost ← node.PATH-COST + problem.ACTION-COST(s, action, s')
    yield NODE(STATE=s', PARENT=node, ACTION=action, PATH-COST=cost)
  
```

Assignment 1

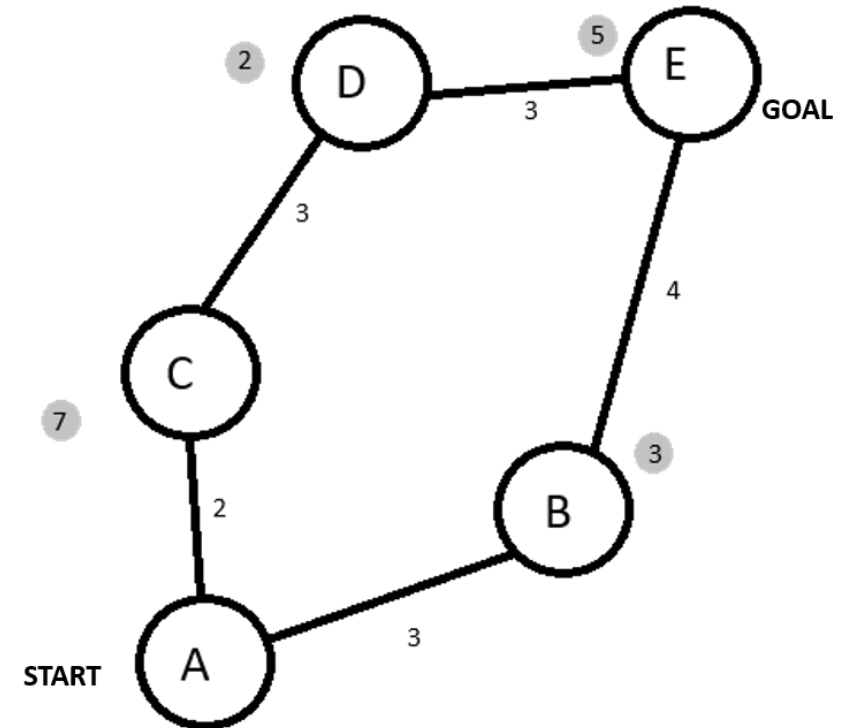
- Not going to spoil the Doom graph for you, but let's see if we agree how this works.
- Greedy best-first search (GBFS).
- Solution:
 - Order of expansion: A, B, (E)
 - Found path: A, B, E
 - Path cost: 7

```

function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node ← NODE(STATE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s] ← child
        add child to frontier
  return failure
  
```

```

function EXPAND(problem, node) yields nodes
  s ← node.STATE
  for each action in problem.ACTIONS(s) do
    s' ← problem.RESULT(s, action)
    cost ← node.PATH-COST + problem.ACTION-COST(s, action, s')
    yield NODE(STATE=s', PARENT=node, ACTION=action, PATH-COST=cost)
  
```



- Example of solution:
 - Order of expansion: S, A, B, (G)
 - Found path: SBG
 - Path cost: 4

Assignment 1

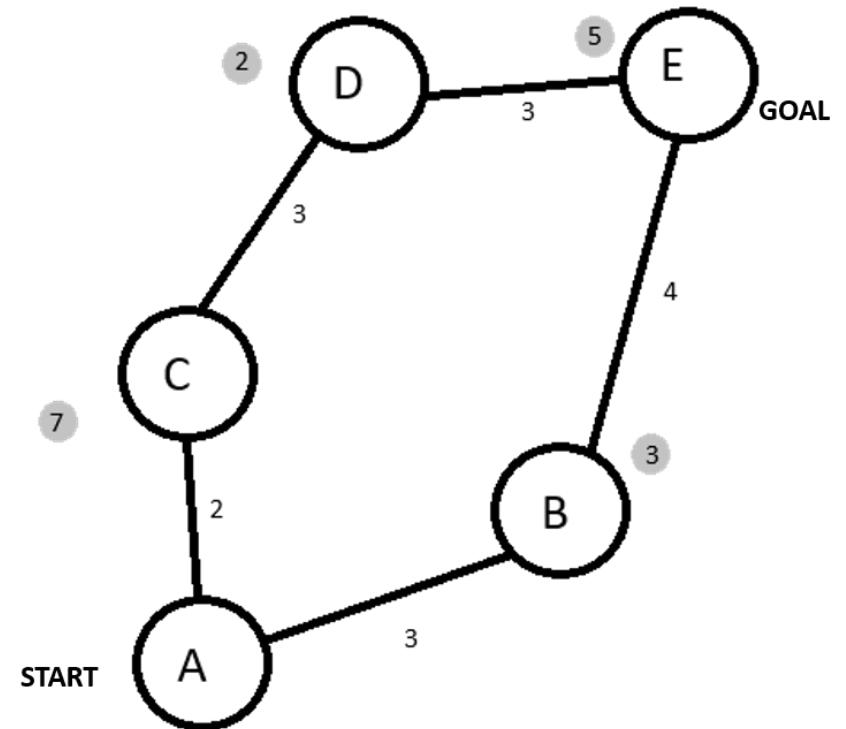
- Not going to spoil the Doom graph for you, but let's see if we agree how this works.
- A* search.
- Solution:
 - Order of expansion: A, B, C, D, (E)
 - Found path: A, B, E
 - Path cost: 7

```

function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node ← NODE(STATE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s] ← child
        add child to frontier
  return failure
  
```

```

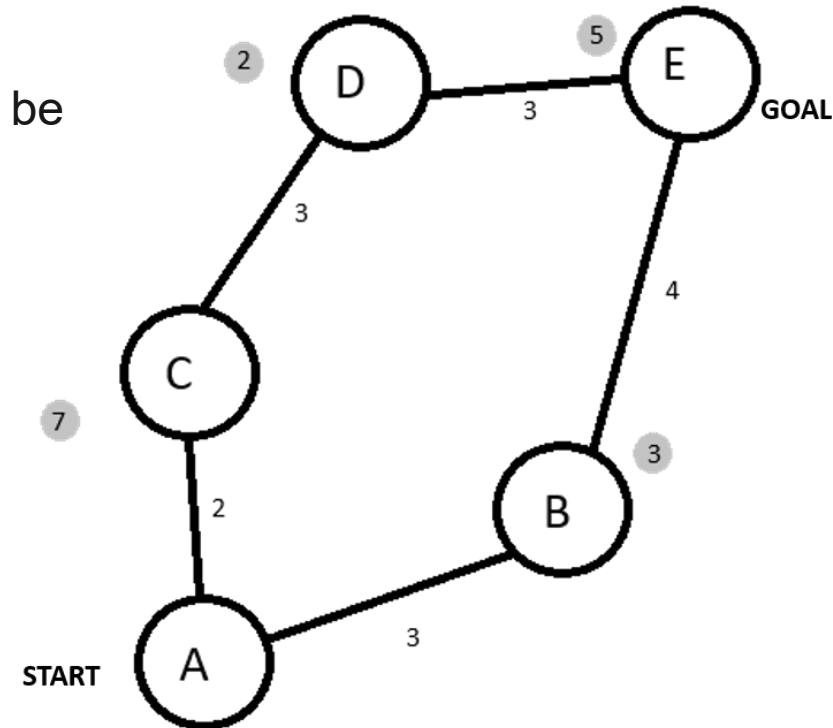
function EXPAND(problem, node) yields nodes
  s ← node.STATE
  for each action in problem.ACTIONS(s) do
    s' ← problem.RESULT(s, action)
    cost ← node.PATH-COST + problem.ACTION-COST(s, action, s')
    yield NODE(STATE=s', PARENT=node, ACTION=action, PATH-COST=cost)
  
```



- Example of solution:
 - Order of expansion: S, A, B, (G)
 - Found path: SBG
 - Path cost: 4

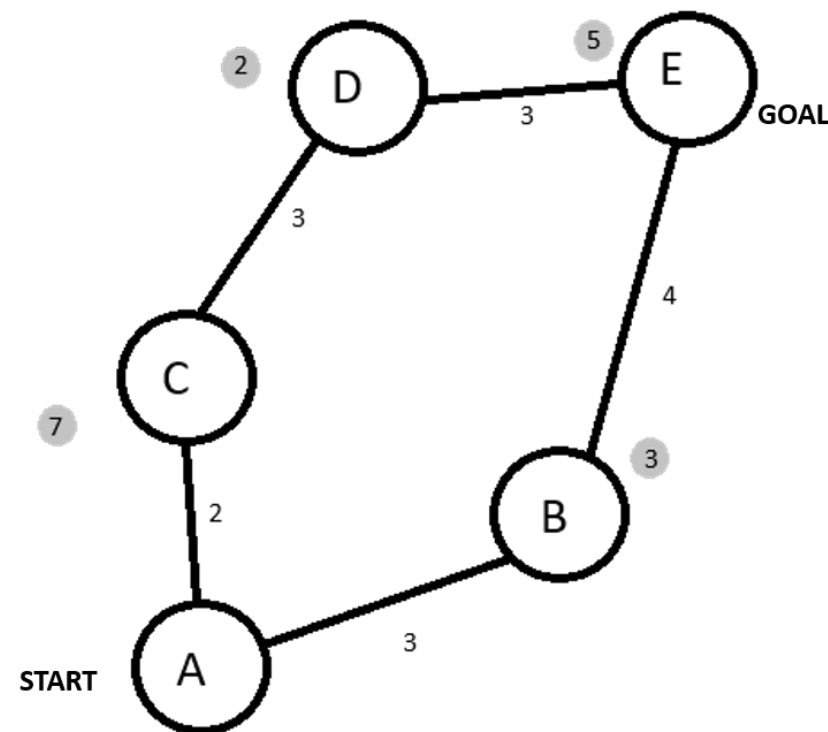
Assignment 1

- The final part of the assignment is about mathematical properties of the heuristic functions which can allow guarantees for the behavior of the algorithms.
- A* search is cost-optimal if arc costs are positive and the heuristic is *admissible* and/or *consistent*. This will be covered in proper detail in the next week's lecture.
- Also see Chapter 3 in the Norvig and Russell book.



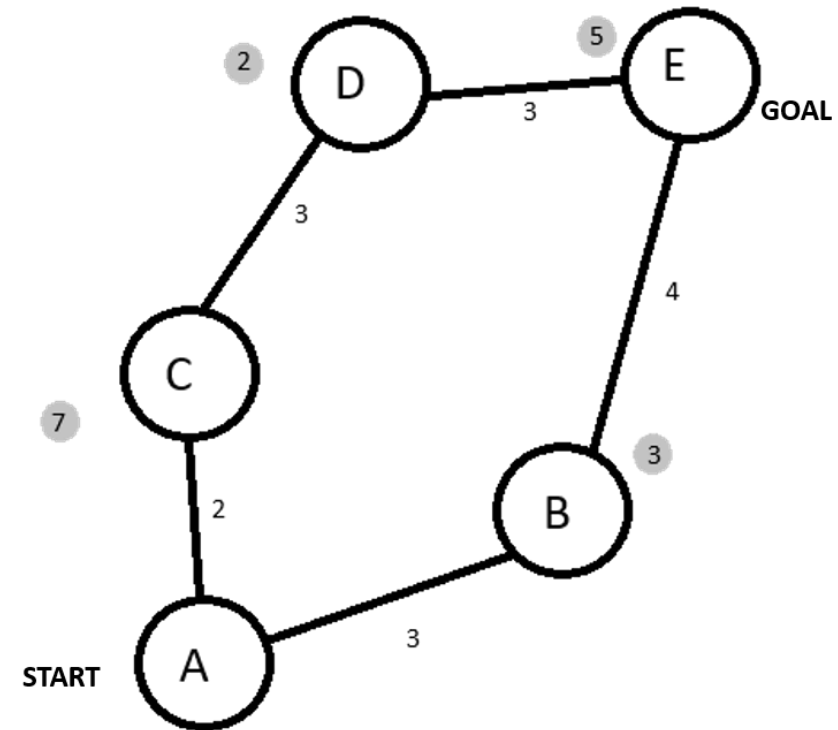
Assignment 1

- A heuristic is *admissible* if it never overestimates the cost from a node to the goal node.
- A heuristic is *consistent* if for every node n and all of its successors n' generated by an action a , we have $h(n) \leq c(n, a, n') + h(n')$.
 - Informally: The estimate of a node should be less than or equal to the estimate of a successor plus the cost of reaching the successor.
 - Even more informally: The heuristic should not overestimate the cost relative to successive estimates.



Assignment 1

- Is our simple graph admissible?
 - No, because $(D, E) = 3$ while $h(e)=5$.
- Is our simple graph consistent?
 - No, because all consistent graphs are admissible.
 - Or $h(d) \leq c(d, a, e) + h(e)$ which gives us $2 \leq 3 + 5$ and this is a contradiction. The graph is not consistent.



Assignment 1

- The bonus task is a bit hard and not something you have to do.
- This goes a bit into why there are so many variants of the algorithms and what can be lost and gained with small changes.
- Tip: If you read the book closely, the answers should be apparent.

2.3.1 Bonus Task

9. How would you have to change the code of the A^* algorithm to make consistency necessary for cost optimality? What may you gain from making the algorithm less robust?

Practical reminders

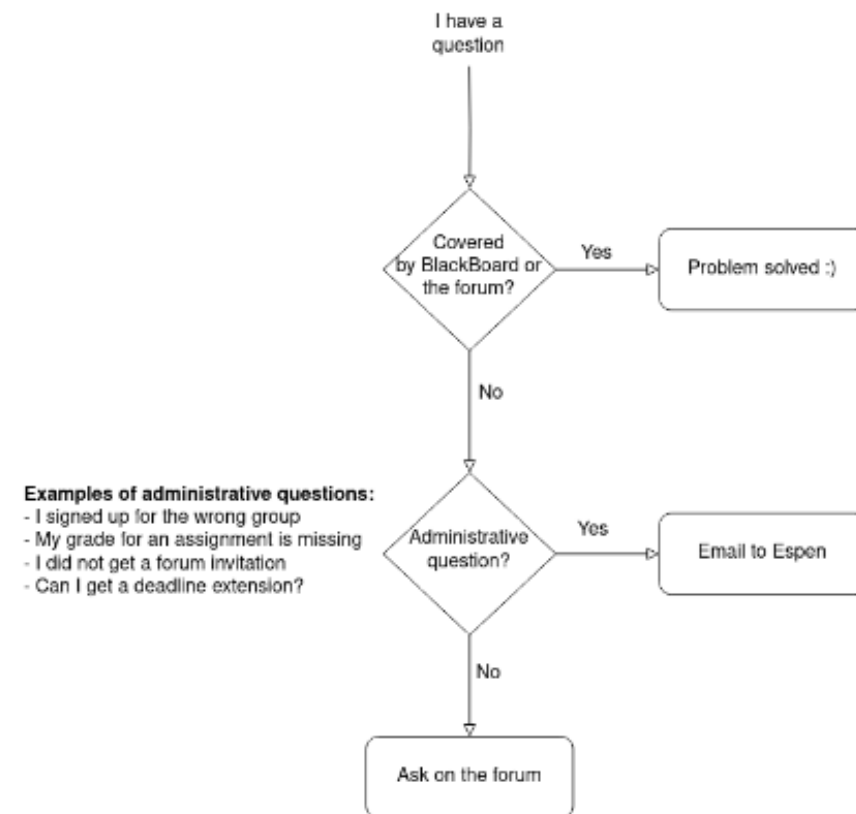
You must upload a **single PDF** containing **typeset** text, equations, and/or diagrams of your solutions.

- Include **natively digital** equations and diagrams. You can as well use hand writing on an tablet as long as it is readable. Do **not** upload scans or photos of hand-written solutions as these will be ignored by the TAs
- You can typeset your equations in Microsoft Word/Google Docs. If you're feeling adventurous, try $\text{\LaTeX}$. [Overleaf](#) is a great place to start

During lab hour periods, lab hours will be arranged:

- Monday - 16:15-18:00. Place: [GL-AO U35](#) ➞
- Tuesday at 14:15 to 16:00. Place: [GL-AO U36](#) ➞
- Wednesday at 08:15 to 10:00. Place: [GL-AO U35](#) ➞
- Fridays at 14:15 to 16:00. Place: [GL-AO U36](#) ➞

Any questions or comments?





For more
information:
www.ntnu.edu



Photo – first slide and this slide: Gløshaugen – Photo: Mentz Indergaard/NTNU
Gjøvik – Photo: Espen Taftø Vestad/NTNU | Ålesund – Photo: Sissel Basso/NTNU

